Name-> Suraj Kumar

Roll NO. -> M25CSA030

CSL 7640: NATURAL LANGUAGE UNDERSTANDING

Assignment-2 REPORT

Q2.) Task 1)

A.) Vanilla RNN

(i) Architecture

Vanilla RNN is a simple recurrent model used for character-level name generation. The model processes the input sequence one character at a time.

Each character is converted into a vector of size **64 (embedding size)**. At each time step, the hidden state is updated using the current input and previous hidden state:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

The hidden state is then passed through a linear layer to predict the next character. The model uses a **single recurrent layer** and does not have any gating mechanism.

(ii) Number of Trainable Parameters

Given:

- Vocabulary size (V) = 5868
- Embedding size (E) = 64
- Hidden size (H) = 64

Embedding:
5868×64=3755525868×64=375552

Input-to-hidden:
64×64=409664×64=4096

Hidden-to-hidden:
64×64=409664×64=4096

Bias:
6464

Output layer:
64×5868=37555264×5868=375552

Output bias:
58685868

Total Parameters

375552+4096+4096+64+375552+5868=765,228375552+4096+4096+64+375552+5868=765,228

Total ≈ 7.65 lakh parameters

(iii) Hyperparameters

- Embedding size: 64
- Hidden size: 64
- Number of layers: 1
- Learning rate: 0.0005
- Optimizer: Adam
- Epochs: 10

(iv) Observation

Vanilla RNN is simple and faster to train but cannot capture long-term dependencies effectively. This may reduce the quality of generated names compared to BLSTM and Attention models.

B.) BLSTM (Bidirectional LSTM)

(i) Architecture

BLSTM is an advanced recurrent model that uses LSTM cells and processes the sequence in both directions.

Each character is converted into a vector of size **64 (embedding size)**. Two LSTM layers are used:

- Forward LSTM (left → right)
- Backward LSTM (right → left)

Each LSTM uses input, forget, and output gates along with a cell state to capture long-term dependencies.

The outputs from both directions are concatenated to form a vector of size **128**. A dropout layer is applied, and then a linear layer predicts the next character.

(ii) Number of Trainable Parameters

Given:

- Vocabulary size (V) = 5868
- Embedding size (E) = 64
- Hidden size (H) = 64

LSTM parameters:

For one LSTM:

- Input weights: 64×256=1638464 *256 = 1638464×256=16384
- Hidden weights: 64×256=1638464 *256 = 1638464×256=16384
- Bias: 256256256

Total = **33024**

For two LSTMs:

2×33024=660482 * 33024 = 660482×33024=66048

Embedding:

5868×64=3755525868 * 64 = 3755525868×64=375552

Output layer:

128×5868+5868=756972128* 5868 + 5868 = 756972128×5868+5868=756972

Total Parameters

66048+375552+756972=1,198,57266048 + 375552 + 756972 = 1,198,57266048+375552+756972=1,198,572

Total ≈ 1.2 million parameters

(iii) Hyperparameters

- Embedding size: 64
- Hidden size: 64
- Number of layers: 1 (per direction)
- Learning rate: 0.0005
- Optimizer: Adam
- Epochs: 10
- Dropout: 0.3

(iv) Observation

BLSTM gives better results than Vanilla RNN because it uses both past and future context. It also handles long-term dependencies better due to LSTM cells. However, it has a large number of parameters and is slower to train.

C.) Attention RNN

(i) Architecture

Attention RNN extends the Vanilla RNN by adding an attention mechanism. The model first processes the input sequence using a basic RNN to generate hidden states for all time steps.

Each character is converted into a vector of size **64 (embedding size)**. A hidden state of size **64** is computed at each step using the RNN.

After that, attention is applied:

- The current hidden state acts as a **query**
- All past hidden states act as **keys and values**
- The model computes attention scores to decide which past positions are important

A **causal mask** is used so the model does not look at future characters.

A context vector is computed as a weighted sum of past states. This context is combined with the current hidden state and passed through a linear layer to predict the next character.

(ii) Number of Trainable Parameters

Given:

- Vocabulary size (V) = 5868
- Embedding size (E) = 64
- Hidden size (H) = 64

Embedding:

5868×64=3755525868 * 64 = 3755525868×64=375552

RNN parameters:

- Input-to-hidden: 64×64=409664 * 64 = 409664×64=4096
- Hidden-to-hidden: 64×64=409664 * 64 = 409664×64=4096
- Bias: 646464

Total = **8256**

Attention (Q, K, V layers):

Each has:

64×64+64=416064 *64 + 64 = 416064×64+64=4160

Total for 3 layers:

3×4160=124803 *4160 = 124803×4160=12480

Output layer:

128×5868+5868=756972128 *5868 + 5868 = 756972128×5868+5868=756972

Total Parameters

375552+8256+12480+756972=1,153,260375552 + 8256 + 12480 + 756972 = 1,153,260375552+8256+12480+756972=1,153,260

Total ≈ 11.5 lakh parameters

(iii) Hyperparameters

- Embedding size: 64
- Hidden size: 64
- Number of layers: 1
- Learning rate: 0.0003
- Optimizer: Adam
- Epochs: 15
- Dropout: 0.3

(iv) Observation

Attention RNN performs better than both Vanilla RNN and BLSTM because it can focus on important parts of the sequence while making predictions. This improves the quality and diversity of generated names. However, it is more computationally expensive.

Q2. Task 2.)

Output Of Code

riya_v21 copy.ipynb | segformer.ipynb | results.txt | biconnected.cu | q1.ipynb | q2.ipynb | TrainingNames.txt

lu2 > q2.ipynb > import torch

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | heysk (Python 3.7.3)

```python
gen = generate_samples(model)

print("Novelty:", compute_novelty(gen, names))
print("Diversity:", compute_diversity(gen))
print("Samples:", gen[:5])
```

Python

```
Training Vanilla RNN...
Epoch 1, Loss: 34532.82
Epoch 2, Loss: 26996.07
Epoch 3, Loss: 24415.59
Epoch 4, Loss: 22740.41
Epoch 5, Loss: 21618.13
Epoch 6, Loss: 20730.94
Epoch 7, Loss: 20051.72
Epoch 8, Loss: 19520.00
Epoch 9, Loss: 18987.90
Epoch 10, Loss: 18570.72
Training BLSTM...
Epoch 1, Loss: 19301.37
Epoch 2, Loss: 3593.41
Epoch 3, Loss: 1336.27
Epoch 4, Loss: 671.00
Epoch 5, Loss: 382.65
Epoch 6, Loss: 217.78
Epoch 7, Loss: 151.65
Epoch 8, Loss: 101.28
Epoch 9, Loss: 73.52
Epoch 10, Loss: 52.80

Training Attention...
Epoch 1, Loss: 37606.86
Epoch 2, Loss: 30542.13
Epoch 3, Loss: 27779.07
Epoch 4, Loss: 26265.04
```


```
Epoch 1, Loss: 37606.86
Epoch 2, Loss: 30542.13
Epoch 3, Loss: 27779.07
Epoch 4, Loss: 26265.04
```


```
Epoch 5, Loss: 25080.90
Epoch 6, Loss: 24321.92
Epoch 7, Loss: 23529.78
Epoch 8, Loss: 22846.84
Epoch 9, Loss: 22256.49
Epoch 10, Loss: 21761.89
Epoch 11, Loss: 21296.13
Epoch 12, Loss: 21019.58
Epoch 13, Loss: 20607.86
Epoch 14, Loss: 20268.13
Epoch 15, Loss: 19993.47

=== VanillaRNN ===
Novelty: 1.0
Diversity: 1.0
Samples: ['kizaa mulrad nair', 'sheetal redwa', 'batesh kumar', 'meeya ali', 'rizala naun gupta']

=== BLSTM ===
Novelty: 1.0
Diversity: 0.96
Samples: ['usiv', 'anum', 'wamy', 'fzuqkun', 'izz']

=== Attention ===
Novelty: 0.995
Diversity: 1.0
Samples: ['raduvatu barju nair', 'fosh zail', 'naka mehta', 'jakhas imehti iyar', 'barsnchmiran boor']
```

Q2.) Task 3

Qualitative Analysis

1. Vanilla RNN

Realism:
The Vanilla RNN generates names that look somewhat realistic but are often inconsistent. Some names resemble real Indian names (e.g., *"sheetal redwa", "batesh kumar"*), but many contain unusual combinations or extra words. The model can learn basic patterns like common surnames but fails to maintain consistency across the full name.

Common Failure Modes:

- Produces extra words or long names unnecessarily
- Weak handling of long sequences
- Repetition or unnatural character combinations
- Limited ability to capture full name structure

Sample Outputs:

- kizaa mulrad nair
- sheetal redwa
- batesh kumar
- meeya ali
- rizala naun gupta

2. BLSTM

Realism:
BLSTM generates shorter and more structured outputs compared to Vanilla RNN. However, many generated names are too short or incomplete (e.g., *"anum", "izz"*). While it captures character patterns better, it sometimes fails to generate full meaningful names.

Common Failure Modes:

- Very short or incomplete names
- Loss of full name structure (missing surname)

- Sometimes generates random character sequences

Sample Outputs:

- usiv
- anum
- wamy
- fzuqkun
- izz

3. Attention RNN

Realism:
Attention RNN produces more realistic and structured names compared to both Vanilla RNN and BLSTM. It is able to generate names with proper first and last name patterns (e.g., *"naka mehta"*). The attention mechanism helps the model focus on relevant parts of the sequence.

Common Failure Modes:

- Occasionally generates slightly distorted names
- Some names contain extra or incorrect characters
- Rare cases of unrealistic combinations

Sample Outputs:

- raduvatu barju nair
- fosh zail
- naka mehta
- jakhas imehti iyar
- barsnchmiran boor

Overall Comparison

- Vanilla RNN: Basic learning, less consistent
- BLSTM: Better sequence understanding but sometimes too short
- Attention RNN: Best performance with more realistic and structured names

---